# MirrorRoomSimulation

Developer Documentation
(accurate as of the current state of the code)

Repository: `Githu228i/MirrorRoomSimulation`

October 12, 2026

**Abstract**

This document is a map of the project for a programmer opening `MirrorRoomSimulation` for the first time. It covers where things live, how the architecture is organized, where the actual computational logic sits (spoiler: *not* in `backendoperators.cpp` — that file is empty), how to extend the code, and what pitfalls exist. It was written by reading the sources, not by reading the original design intent.

## Contents

# 1 About the project

`MirrorRoomSimulation` is an educational / demo C++ application with a Qt 6 (Widgets) GUI. The user clicks with the mouse to build a closed (or open) chain of mirrors inside a "room", edits vertices and mirror types through dialogs, and can then trace a ray bouncing off the mirrors.

## 1.1 Tech stack

| Component | Purpose |
|---|---|
| C++17 | Language. |
| Qt 6 (Widgets) | GUI, `QPainter`, dialogs, signals/slots. |
| CMake $\geq$ 3.16 | Build system. |
| Qt Creator | Recommended IDE. |
| `clangd` | Optional, for navigation. |

## 1.2 Quick start

```
cmake -S . -B build/Desktop-Debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/Desktop-Debug -j
./build/Desktop-Debug/MirrorRoomSimulation
```

Listing 1: Build

**Important.** `build/`, `.qtcreator/`, `CMakeLists.txt.user`, `*.back*`, `.cache/` must **not** be committed to Git. See **??**.

# 2 Repository structure

| File | Purpose |
|---|---|
| `main.cpp` | `QApplication`, translator setup, launches `MainWindow`. |
| `mainwindow.h/.cpp` | Main window. Mouse handling, `paintEvent`, dialog invocation. |
| `mainwindow.ui` | Empty design file: just `centralwidget`, `menubar`, `statusbar`. All rendering is done manually in `paintEvent`. |
| `points.h/.cpp` | `Points` (base point) and `Beam` (ray: point + angle). |
| `mirror.h/.cpp` | `Mirror` class hierarchy and its subclasses. Mirror intersections. Drawing. |
| `polygon.h/.cpp` | `Polygon`: a set of mirrors, closure checks, validation, ray tracing. |
| `frontendoperators.h/.cpp` | Scene manager: `CreatePoint`, `ConnectPoints`, `ChangeMirror`. |
| `backendoperators.h/.cpp` | **Empty stub.** Constructor only. No logic. |
| `changeorconnectdialog.h/.cpp` | Dialog with two buttons: "Edit" / "Connect". |
| `changepointdialog.h/.cpp` | Vertex editing dialog. Defines `ChangePointData`. |
| `MirrorRoomSimulation_ru_RU.ts` | Translation file (empty). |
| `README.md` | One line. |
| `CMakeLists.txt` | Build. |

**Orientation tip.** If you are looking for "where the backend logic is" — go to `polygon.cpp` (ray tracing) and `mirror.cpp` (intersections). `backendoperators.cpp` is a skeleton that was apparently meant to be filled in but never was.

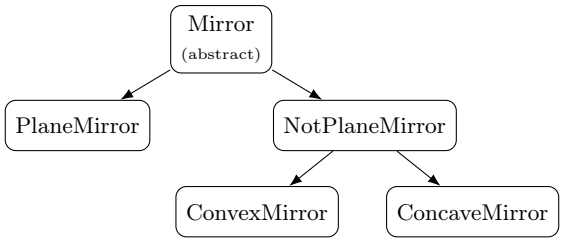
# 3 Architecture

## 3.1 Layers

1. **Geometry primitive:** `Points`, `Beam`.
2. **Mirror model:** `Mirror` and subclasses + intersection algorithms (inside an anonymous namespace in `mirror.cpp`).
3. **Scene composition:** `Polygon` — holds the vector of mirrors, traces rays, validates the contour.
4. **Scene manager:** `FrontEndOperators` — a wrapper over `Polygon` for UI operations.
5. **GUI:** `MainWindow` + dialogs.

**Rule.** The model (`Points`, `Mirror`, `Polygon`) must not know about `QMainWindow`. However, `Polygon` already knows about `QPainter` — `Mirror` subclasses know how to draw themselves. This is a deliberate simplification (see **??**).

## 3.2 Class hierarchy




- `Mirror` — abstract base. Fields: `start_`, `end_`, `rad_`. Virtual destructor is mandatory — stored as `std::unique_ptr<Mirror>`.
- `PlaneMirror` — flat mirror (rad = 0). Overrides `isFlat()` → `true`.
- `NotPlaneMirror` — intermediate class with a protected constructor.
- `ConvexMirror` — `rad > 0`, circle center "left" of the chord.
- `ConcaveMirror` — `rad < 0`, center "right".

## 3.3 Data flow

1. User clicks in `MainWindow::mousePressEvent`.
2. `MainWindow` decides: hit a vertex, too close, new vertex.
3. It calls one of `FrontEndOperators`: `CreatePoint`, `ConnectPoints`, or `ChangeMirror`.
4. That method manipulates `Polygon::sides_` (`std::vector<std::unique_ptr<Mirror»`).
5. `MainWindow` calls `update()` — repaint.
6. `MainWindow::paintEvent` draws `first` and all mirrors via their virtual `Draw()`.

# 4 Key entities

## 4.1 Points and Beam (points.h)

```
class Points {
public:
    Points(double x = 0.0, double y = 0.0);
    virtual ~Points() = default;
    double getX() const;
    double getY() const;
    double Dist(const Points& other) const;
    virtual std::string Cords() const;
protected:
    double x_, y_;
};

class Beam : public Points {
public:
    Beam(double x = 0.0, double y = 0.0, double angle = 0.0);
    double getA() const;
    std::string Cords() const override;
private:
    double angle_;
};
```

Listing 2: Points / Beam

`Cords()` returns a string of the form `"x y"` (or `"x y angle"` for a beam) with 3 decimal places. It is only used for debug file output.

## 4.2 The Mirror hierarchy (mirror.h)

Key `Mirror` methods:

| Method | Purpose |
|---|---|
| `Mirror(st, end, rad)` | Throws `std::invalid_argument` on zero chord or $\|rad\| < d/2$. |
| `isFlat()` | `true` if $\|rad\| < 10^{-9}$ or $> 10^{7}$. |
| `CircleCenter()` | Circle center; side determined by the sign of `rad`. |
| `CircleCenter(bool leftSide)` | Explicit side selection. |
| `PointOnArc(p, c)` | Whether a point lies on the arc/segment. |
| `Intersections(other)` | Vector of intersection points. |
| `Touches(other)` | `!Intersections().empty()`. |
| `Crossing(other)` | Intersect at more than just shared endpoints. |
| `Draw(painter, insideOnLeft)` | Virtual. Pure virtual in the base. |

Meaning of the `rad_` sign:

- $rad > 0$ — center *left* of the direction `start → end`. Corresponds to `ConvexMirror`.
- $rad < 0$ — center *right*. Corresponds to `ConcaveMirror`.
- $rad = 0$ — segment (`PlaneMirror`).

## 4.3 Polygon (polygon.h)

The scene container.

```cpp
class Polygon {
    // ...
private:
    std::vector<std::unique_ptr<Mirror>> sides_;
    Points first;              // first point of an open contour
    bool isExist_ = false;     // whether the first point exists
    bool isCycled_ = false;    // whether the contour is closed
    Beam beam_;                // beam for tracing
};
```

Listing 3: Polygon fields

Key methods:

| Method | What it does |
|---|---|
| `Push(const Mirror&)` | Copies the mirror via `dynamic_cast` and appends to `sides_`. Calls `Write()`. |
| `Del(int idx)` | Removes a mirror by index. Calls `Write()`. |
| `Check()` | Validation: contour is closed, sides intersect only at shared vertices of neighbors. |
| `TraceBeam(maxBounces)` | Ray tracing. Writes `beam.txt`. |
| `Write()` | Dumps vertices and beam to `points.txt`. |

**Side effect.** `Push`, `Del`, `Write`, `TraceBeam` open files. Under heavy editing this can slow things down and clutter the working directory.

## 4.4 FrontEndOperators (`frontendoperators.h`)

```cpp
class FrontEndOperators {
public:
    FrontEndOperators();
    void CreatePoint(double x, double y);
    void ConnectPoints(int num);
    void ChangeMirror(int num, const ChangePointData &data);
    friend class MainWindow;
private:
    Polygon prototype;
};
```

Listing 4: FrontEndOperators

**Misleading name.** This is a *scene manager*, not a rendering layer. Rendering lives in `MainWindow::paintEvent` and `Mirror::Draw`.

Method logic:

- `CreatePoint` — adds a new point. If `isExist_` is false, stores `first` and sets the flag. Otherwise, builds a `PlaneMirror` from the end of the last mirror to the new point. Limit: 8 points (`if (prototype.sides_.size() < 8)`).
- `ConnectPoints(num)` — closes the contour: erases the mirrors before `num`, adds a final mirror from `end` to `start`, sets `isCycled_ = true`.
- `ChangeMirror(num, data)` — the most complex function. Handles cases: last point of an open contour, first point of a closed one, regular point. Everywhere it uses `dynamic_cast` to determine the mirror type and recreates adjacent mirrors.

### 4.5 MainWindow (mainwindow.h)

Only two fields:

```
Ui::MainWindow *ui;
FrontEndOperators polygonCreator;
```

Logic:

- `mousePressEvent` — all click handling. Hardcoded: `y > 1000` is ignored, hit radius `25`, minimum distance between points `100`.
- `paintEvent` — draws `first`, the line from `first` to `temp[0]->getStart()`, and all mirrors.

### 4.6 Dialogs

`ChangeOrConnectDialog` — simple two-button window (`accept` / `reject`). Placeholder text: `" "` ("your text goes here").

`ChangePointDialog` — single-vertex editing. Returns:

```
struct ChangePointData {
    double x;
    double y;
    int mirrorType;    // 0 = plane, 1 = concave, 2 = convex
    double radius;
};
```

If `lastPoint == true`, the mirror-type combo box and the radius field are disabled (the last vertex has no mirror yet).

### 4.7 BackEndOperators — an empty stub

```
#include "backendoperators.h"
BackEndOperators::BackEndOperators() {}
```

Listing 5: The entire backendoperators.cpp

> **Important.** Do not look for logic here. There is none. All computational code lives in `mirror.cpp` and `polygon.cpp` inside unnamed `namespace { }` blocks. If you want to add something "to the backend", either fill in `BackEndOperators` or create a new module — do not keep spreading logic across `polygon.cpp`.

## 5 User interaction

### 5.1 Creating a point

Click on empty space:

1. If you hit an existing point ($\leq 25$) — edit it.
2. If too close to an existing point ($\leq 100$) — message " !" ("Points too close!").
3. Otherwise — `CreatePoint(x, y)`.

Limit: **8 sides**.

### 5.2 Editing a point

Click on an existing point:

- **Contour closed** — `ChangePointDialog` opens directly.
- **Contour open** — `ChangeOrConnectDialog` opens first ("Edit" or "Connect").

### 5.3 Connecting points

Choosing "Connect" in an open contour:

- The last mirror cannot be connected — `QMessageBox::critical`.
- Otherwise `ConnectPoints(i)`: erases all mirrors before `i`, adds one closing mirror.

## 6 Geometric algorithms

### 6.1 Intersections (`mirror.cpp`, anonymous namespace)

- `IntersectSegments` — segment/segment, including the collinear case with overlap computation.
- `IntersectSegmentArc` — segment/arc, quadratic equation.
- `IntersectArcs` — arc/arc, standard two-circle intersection with `PointOnArc` verification.

Constants: `EPS = 1e-9`, `POINT_EPS = 1e-6`, `PAR_EPS = 1e-10`, `FLAT_R = 1e7`.

### 6.2 Ray tracing (`polygon.cpp`)

`TraceBeam`:

1. Position and direction come from `beam_`.
2. For each mirror — ray intersection.
3. Pick the nearest with $t > 10^{-7}$.
4. Compute the surface normal at the hit point (`GetNormal`).
5. Reflect: `d' = d - 2(d·n)n` (`ReflectDirection`), renormalized.
6. Append to `beam.txt`: hit positions space-separated.

## 7 File I/O

| File | Written by | Format |
|---|---|---|
| `points.txt` | `Polygon::Write()` | Header, vertex coords, beam. |
| `beam.txt` | `Polygon::TraceBeam()` | x y for each hit. |

**Warning.** Both files are written to the current working directory (not `$HOME`, not the project directory). When run from Qt Creator this is `build/Desktop-Debug/`. Easy to commit by accident — keep them in `.gitignore`.

# 8 Build and CMake

```
set(PROJECT_SOURCES
    main.cpp
    mainwindow.cpp mainwindow.h mainwindow.ui
    mirror.h mirror.cpp
    points.h points.cpp
    frontendoperators.h frontendoperators.cpp
    backendoperators.h backendoperators.cpp
    polygon.h polygon.cpp
)
qt_add_executable(MirrorRoomSimulation
    ${PROJECT_SOURCES}
    changeorconnectdialog.h changeorconnectdialog.cpp
    changepointdialog.h changepointdialog.cpp
)
```

Listing 6: CMakeLists.txt, fragment

**Oddity.** The dialogs are added in a separate block of `qt_add_executable` rather than inside `PROJECT_SOURCES`. If you add a new file — pick one place and follow the existing style.

## 8.1 Translations

```
lupdate . -ts MirrorRoomSimulation_ru_RU.ts
lrelease MirrorRoomSimulation_ru_RU.ts
```

**Bug.** In `main.cpp` the translation name is `"MirrorRoomSimulationProject_" + locale`, while CMake generates `MirrorRoomSimulation_`. The `Project` prefix is extraneous. Either change `main.cpp` to `"MirrorRoomSimulation_"` or rename `TS_FILES`. Also, the `.ts` file is empty — nothing to translate yet.

# 9 Extension points

| Task | Where to go |
|---|---|
| Add a new mirror type | Subclass `Mirror` + `Draw`. Plus: constructor in `Polygon::Push`, `ChangeMirror`, and every `dynamic_cast` branch. |
| Change reflection physics | `ReflectDirection` and `GetNormal` in `polygon.cpp` (anonymous namespace). |
| Add a new geometry primitive | `Points` or a separate class. |
| Change rendering | `Mirror::Draw` in subclasses, `drawCircularArc` in `mirror.cpp`, `MainWindow::paintEvent`. |
| Add a menu item / hotkey | `mainwindow.ui` (empty) or `MainWindow` in code. |
| Remove the 8-point limit | `FrontEndOperators::CreatePoint`, the `sides_.size() < 8` condition. |
| Configure tracing (bounce count) | Parameter `maxBounces` of `TraceBeam`. Currently not invoked from the UI. |

## 9.1 Checklist for adding a file

1. Create `.h` and `.cpp`.

2. Add to `CMakeLists.txt` (either in `PROJECT_SOURCES` or next to the dialogs — but not both).
3. If the class is a `QObject`, add `Q_OBJECT`.
4. If it contains translatable strings — `tr(...)`.
5. Build; make sure there are no moc/uic warnings.
6. Commit only sources and CMake, never `build/`.

# 10 Known issues and oddities

| What | Where / why |
|---|---|
| `BackEndOperators` is empty | `backendoperators.cpp`. Logic is scattered across `polygon.cpp` and `mirror.cpp`. |
| Translation name mismatch | `main.cpp` looks for `MirrorRoomSimulationProject_*`, CMake generates `MirrorRoomSimulation_*`. |
| Magic numbers | `1600`, `1200`, `1000`, `25`, `100`, `8`, `4`. All in `mainwindow.cpp` and `frontendoperators.cpp`. |
| `Polygon::Write()` on every `Push`/`Del` | Opens `points.txt` on every change. |
| `dynamic_cast` chains | `Polygon::Push`, `FrontEndOperators::ChangeMirror`. Adding a new mirror type forces edits everywhere. |
| `MainWindow` as friend | In `Polygon` and `FrontEndOperators`. Breaks encapsulation. |
| `mirror.cpp` includes `QPainter` | Model knows about the GUI. Layer mixing. |
| Placeholder text in the dialog | `"  "` in `changeorconnectdialog.cpp`. |
| `README.md` is empty | One line only. |
| `ChangePointData` in `changepointdialog.h` | `FrontEndOperators` pulls a dialog header only for the struct. Better moved to `changepointdata.h`. |

# 11 Git rules

## 11.1 What not to commit

- `build/`, `build-*/`, `cmake-build-*/`;
- `CMakeLists.txt.user`, `*.user`, `.qtcreator/`;
- `.cache/`, `compile_commands.json`;
- `*.back`, `*.back1`, `*.back2`, `* `, `*.orig`, `*.rej`;
- `points.txt`, `beam.txt` (runtime artifacts).

```
build/
build-*/
cmake-build-*/
CMakeLists.txt.user
*.user
```

```
*.user.*
.qtcreator/
.cache/
compile_commands.json
*.back
*.back1
*.back2
*.back3
*~
*.orig
*.rej
points.txt
beam.txt
```

Listing 7: .gitignore

### 11.2 If junk is already in the index

```
git rm -r --cached build/ .qtcreator/ CMakeLists.txt.user
git rm --cached points.txt beam.txt
git commit -m "Stop tracking build artifacts and IDE files"
```

### 11.3 Branch strategy

Branches `main` and `backend` have *different file structures*. `git merge` between them yields dozens of "modify/delete" conflicts and makes little sense.

- If `backend` is the current version:

```
git branch main-backup main      # backup
git checkout backend
git branch -f main backend
git checkout main
git push --force-with-lease origin main
```

- If you need to bring over individual commits — `git cherry-pick <hash>`.
- Avoid `git merge` between branches with different file structures: Git does not map `mirror.cpp` to `backendoperators.cpp` as the same module.

## 12 Code conventions

- Classes — `CamelCase`; methods — `camelCase`; private fields — `camelCase_` (e.g., `sides_`, `rad_`).
- Headers — include guards, not `#pragma once` (the project uses guards everywhere).
- Do not use `using namespace std;`.
- Prefer forward declarations in `.h` (`class QPainter;`) over `#include <QPainter>` — as in `mirror.h`.
- Public methods should have a short comment about preconditions (as `Mirror` does).
- New mirror types — subclass `Mirror`, implement the pure virtual `Draw`.

## 13 Debugging and common problems

| Symptom | What to do |
|---|---|
| `undefined reference to vtable` | Missing `Q_OBJECT` (for QObject classes) or the file is not in CMake. |
| `moc_*.cpp` not found | Check `AUTOMOC ON`. |
| A new point is not drawn | `update()` was not called after modifying the model. |
| Scene not repainted after `ChangeMirror` | Check the return paths — `ChangeMirror` returns early in several branches. |
| `std::invalid_argument` from `Mirror` | Radius smaller than half the chord, or a zero chord. |
| Translations not applied | Prefix `MirrorRoomSimulationProject_` in `main.cpp` does not match CMake. See the "Translations" section. |
| `points.txt` appears in the working dir | That is `Polygon::Write()`. Add it to `.gitignore`. |
| Qt Creator creates `*.back1/.back2` | Add to `.gitignore`, remove with `rm -f *.back*`. |

## 14 Cheat sheet

```
# Build
cmake --build build/Desktop-Debug -j

# Clean rebuild
rm -rf build/Desktop-Debug && \
  cmake -S . -B build/Desktop-Debug && \
  cmake --build build/Desktop-Debug -j

# Clean up Qt Creator junk and runtime files
rm -f *.back *.back1 *.back2 *.back3 *~ points.txt beam.txt

# Verify junk is not in the index
git ls-files | grep -E '^(build/|\.qtcreator/)' && echo "FOUND" || echo "clean"

# Update translations
lupdate . -ts MirrorRoomSimulation_ru_RU.ts
lrelease MirrorRoomSimulation_ru_RU.ts
```

Listing 8: Common commands

## 15 Where to look first

1. `main.cpp` — entry point.
2. `mainwindow.h` — window state.
3. `mainwindow.cpp` — `mousePressEvent` (main user input) and `paintEvent` (rendering).
4. `frontendoperators.h` — scene operations.
5. `frontendoperators.cpp` — `CreatePoint`, `ConnectPoints`, `ChangeMirror`.
6. `polygon.h` — scene model.
7. `polygon.cpp` — `TraceBeam`, reflection.

8. `mirror.h` — mirror hierarchy.
9. `mirror.cpp` — intersections and `Draw`.
10. Dialogs — as needed.

---

*This document was written by reading the actual sources. When the structure changes, update sections 2, 3, 9, and 10.*